

E00 · 기호 사전 — 이 책에 나오는 모든 글자

이 권은 Sleep-Time Compute 모노그래프의 입구이자 참조 사전이다. 본문을 요약하지 않는다. 본문이 쓰는 글자를 하나씩 꺼내 **무엇인가 · 어디에 사는가 · 무엇을 바꾸는가 · 어느 장이 그 뜻을 소유하는가** 네 항목으로 설명한다. 이 권을 덮은 독자는 본문의 수식을 풀지는 못해도 읽을 수는 있어야 한다. 그것이 이 권의 목표 전부다.

대상 독자는 transformer inference와 서버는 잘 알지만 (KV cache, prefill/decode, batching, SLA) **training**은 해 본적이 없고 이 라인의 논문도 처음인 엔지니어다. 그래서 gradient·momentum 같은 말도 여기서 한 줄로 풀어 쓴다. 유도와 증명은 하나도 하지 않고 전부 본문으로 넘긴다.

0.1 글자가 필요한 이유

이 책이 다루는 절차는 한 문장으로 줄어든다.

핵심 질의가 도착하기 전에, 놓고 있는 계산 자원으로 미리 무언가를 계산해서 **상태를 바꿔 둔다**. 그러면 나중에 오는 질의가 더싸지거나 더 정확해진다. 이것이 sleep-time compute 이다.

문제는 “상태”가 한 가지가 아니라는 것이다. 본문이 나란히 놓는 세 논문은 전부 자기 절차를 sleep이라 부르는데, 하나는 모델 **밖의 텍스트**를 고치고 (→ ch14), 하나는 **배포되는 가중치**를 고치고 (→ ch21), 하나는 둘 중 어느 것도 아닌 세 번째 것을 고친다 (→ ch17). 무엇이 바뀌었는지 주목하는 어휘가 없으면 이 셋은 비교 대상조차 되지 못한다. 그래서 기호는 장식이 아니라 **비교를 가능하게 하는 최소 장치**다.

이 권에서 익혀야 할 것은 딱 세 묶음이다. **상태 세층** ($\Theta \cdot W \cdot E$), **비용 내값** ($B_s \cdot L_w \cdot C_{cap} \cdot \rho$), 그리고 그 둘을 잇는 **다섯 개의 식**이다. 나머지 글자는 전부 이 세 묶음의 부품이다.

자세한 것은 ch01 § 01.1.

0.2 상태 세 층 — Θ, W, E

세 층은 크기순도 정확도순도 아니다. 가르는 기준은 하나다 — **누가 소유하고 언제 사라지는가**.

기호 풀이 Θ (대문자 세타, slow weights). **무엇인가**: 모델파일에 들어 있는 그 가중치. 당신이 배포하는 아티팩트를 이루는 숫자전부다. **어디에 사는가**: 가중치 파일, 그리고 서버 중에는 GPU의 HBM. 모든 요청이 공유한다. **무엇을 바꾸는가**: 이것이 바뀌면 **배포물이 바뀐다**. 되돌리려면 이전 checkpoint로 롤백하는 수밖에 없다 — Θ 에는 “지우기”라는 연산이 아예 정의되어 있지 않다. **소유 장**: ch01 § 01.2.

기호 풀이 W (fast weights). **무엇인가**: 추론 도중에 토큰단위로 값이 움직이는 가중치 형태의 내부 상태. 가중치처럼 생겼지만 배포물이 아니라 요청에 딸린 상태다. **어디에**

사는가: 그요청·세션이 잡고 있는 메모리. 크기는 문맥 길이와 무관하게 고정이다. **무엇을 바꾸는가:** 상태를 들고 있는 주체가 사라지면 함께 사라진다. 그래서 롤백 단위가 세션이다. **소유장:** ch01 § 01.2.

기호 풀이 E (external store). **무엇인가:** 모델 밖에 텍스트·벡터·그래프로 저장해 두는 기억. 당신의 어휘로는 vector DB 한 벌, 또는 사용자별 메모 테이블 한 벌이다. **어디에 사는가:** 모델 밖 저장소. 요청과 독립적으로 존속한다. **무엇을 바꾸는가:** 임의 시점에 레코드 하나 단위로 지울 수 있고, 재계산이 가능하므로 evict가 정책 문제로 남는다. **소유장:** ch01 § 01.2.

표 0-1 — 세 층을 가르는 것은 “언제 사라지는가” 다

기호	이름	어디에 사는가	지우는 방법	롤백 단위
Θ	slow weights	가중치 파일·HBM	없음 (새아티팩트를 배포)	checkpoint
W	fast weights	요청·세션 상태	세션 종료	세션
E	external store	모델 밖 저장소	레코드 삭제	레코드

비유 회사의 세 가지 기록에 대응시킬 수 있다. Θ 는 **사규**다 — 모두가 따르고, 고치려면 개정판을 다시 배포해야 하며, “이 조항만 오늘저녁까지 지워 두자”가 불가능하다. W 는 **오늘 회의 중 머릿속에 쌓이는 맥락**이다 — 회의가 끝나면 사라지고, 아무리 긴 회의여도 머릿용량은 그대로다. E 는 **캐비닛의 문서철**이다 — 회의와 무관하게 남아 있고, 한 장만 빼서 버릴 수 있다.

한계. 이 비유는 **수명과 삭제 가능성에서만** 맞다. 크기와 정보량에서는 깨진다 — 사규가 문서철보다 얇다는 감각을 그대로 옮기면 안된다. Θ 가 담을 수 있는 정보량은 파라미터 수와 dtype이 정하며, 그 상한 자체가 본문 한 장의 주제다 (→ ch09).

시스템 모델링 관점 이 3분할은 새 이론이 아니라 **회계 단위의 선택**이다. 당신이 이미 weights와 KV cache와 vector DB를 서로 다른 예산 항목으로 관리하는 것과 같은 이유다. 나누고 나면 서빙 질문 네 개를 비로소 물을 수 있다. (1) **evict해도 되는가** — E 는 정책, W 는 세션 종료와 함께 자동, Θ 는 불가. (2) **배치에 섞어도 되는가** — E 의 산출물은 prompt에 붙으므로 섞이고, 사용자별 Θ delta가 생기면 shared-weight batching의 전제가 깨진다. (3) **100번 반복하면 무엇이 나빠지는가** — E 쪽은 저장량이 자라고 검색 정밀도가 떨어지며, Θ 쪽은 이전에 알던 것이 손상된다. **열화의 정체**가 층마다 다르다. (4) **롤백 단위는 무엇인가** — 표 0-1의 마지막 열이 답이다.

자세한 것은 ch01 § 01.1 – § 01.2.

0.3 읽기는 한 줄뿐 — 식 (R)

답을 만들 때 모델이 무엇을 보는지를 남김없이 적은 식이다.

$$\hat{y} = f(q; \Theta, W, \text{ret}(E, q)) \quad (\text{R})$$

직관 이 식이 말하는 것은 “답 $\hat{y}$ 는 질문 q 를 받아, 세 층을 전부 보고 만들어진다”이다. 그뿐이다. 세미콜론 왼쪽이 이번에 들어온 것이고, 오른쪽이 이미 갖고 있던 것이다.

기호 풀이 q = 질의 (질문 하나, 벡터로도 쓰고 요청 전체를 가리키기도 한다). $\hat{y}$ = 모델이 내놓는 답. 모자 기호는 “생성된 것 · 추정된 것”을 뜻한다. f = 모델의 읽기 함수, 즉 당신이 아는 forward pass 전체. $\text{ret}(E, q)$ = 질의 q 로 external store E 를 조회하는 연산 — 당신의 어휘로는 **vector DB 조회 한 번**이고, 비용도 그렇게 회계한다.

읽기가 한 줄인데 쓰기가 세 줄인 것이 이 책의 구조 그 자체다. 읽을 때는 세 층이 구분 없이 합쳐지지만, **쓸 때는 어느 층에 쓰는지**가 전부를 결정한다.

자세한 것은 ch01 § 01.3.

0.4 쓰기는 세 줄 — 식 (U-W), (U-Θ), (U-E)

세 식의 차이는 전부 **좌변**에 있다. 그래서 어떤 논문을 읽든 첫질문은 하나다: 이 절차의 좌변은 무엇인가.

W 에 쓰기 — 식 (U-W)

$$W_t = \alpha_t W_{t-1} + S_t, \quad S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t) \quad (\text{U-W})$$

직관 이 식이 말하는 것은 “**토큰이 하나 도착할 때마다** W 를 조금 고쳐 쓴다”이다. 고치는 방향은 “지금 이 토큰을 얼마나 못 맞혔는가” 가정하고, 고치고 나면 옛 내용은 조금 바랜다.

기호 풀이 t = 토큰 (또는 chunk) 번호. k_t, v_t = 이번 토큰에서 뽑은 key 와 value — “이 열쇠를 넣으면 이 값이 나와야 한다”는 한 쌍이다. $\ell(\cdot; \cdot)$ = **inner loss** (소문자 엘). 지금 상태 W 에 k_t 를 넣었을 때 v_t 가 나오는지 재는 **오차측정기**다. $\nabla_W \ell$ = 그 오차가 가장 커지는 방향, 즉 **gradient**. 앞에 빼기가 붙어 있으므로 실제 움직임은 오차가 줄어드는 쪽 한 걸음이다. η_t = 그 걸음의 크기 (**inner learning rate**). S_t = 지금까지 걸어온 방향의 관성 (**momentum**), β_t = 그 관성을 얼마나 유지할지. $\alpha_t \in [0, 1]$ = **retention gate**, 옛 상태를 얼마나 남길지의 비율이다 (1 이면 전부 유지, 0 이면 소거).

이 식은 Neural Memory 모노그래프가 300 쪽에 걸쳐 해부한 그 식이며, 이 책은 부호 하나 바꾸지 않고 물려받는다. 유도는 그쪽에 있고 여기서는 읽는 법만 있으면 된다.

비유 오답노트다. 문제를 풀 때마다 틀린 만큼만 노트를 고쳐 쓰고, 최근에 자주 틀린 유형 쪽으로 손이 계속 쏠린다 — 그 쏠림이 momentum 이다.

한계. 오답노트는 지면을 계속 덧붙이지만 W 는 크기가 고정이다. 새로 쓴 만큼 옛

내용이 덮인다. W 는 무손실 기록이 아니라 고정 크기 손실 압축이며, KV cache를 append-only 무손실 목록으로 아는 감각을 그대로 옮기면 틀린다.

Θ 에 쓰기 — 식 (U- Θ)

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s) \quad (\text{U-}\Theta)$$

직관 이 식이 말하는 것은 “모아 둔 경험으로 학습 데이터를 만들고, 그것으로 학습을 한 라운드 돌린다”이다. 실무 언어로는 fine-tuning job 한 번이고, 끝나면 새 가중치 파일이 하나 나온다.

기호 풀이 아래첨자 k = **sleep 라운드** 번호. 토큰 번호 t 와 섞지 않는다. $\mathcal{H}_k$ = 그 라운드까지 쌓인 경험 (대화로그 · 실패 사례 · 문서). $\text{gen}(\cdot; B_s)$ = 예산 B_s 를 써서 경험을 학습 가능한 집합으로 바꾸는 절차 — replay · distillation · 합성 데이터 생성이 전부여기 들어간다. $\mathcal{R}_k$ = 그 결과인 학습 집합. $\mathcal{L}$ = **outer loss** (대문자), 과제 자체의 손실. $\nabla_{\Theta} \mathcal{L}$ = Θ 를 어디로 옮겨야 그 손실이 줄어드는지. η_{Θ} = 그 걸음의 크기 (**sleep learning rate**)이며 η_t 와 다른 값이다.

주의 맨 k 는 sleep 라운드 번호이고, key 벡터는 언제나 첨자를 달아 k_t 로 쓴다. 첨자 유무가 두 글자를 가르다.

E 에 쓰기 — 식 (U-E)

$$E_{k+1} = \text{wr}(E_k, \hat{c}_k), \quad \hat{c}_k = S(c_k; B_s) \quad (\text{U-E})$$

직관 이 식이 말하는 것은 “질의를 보기 전에 문맥을 미리 읽고 추론까지 해서, 그 결과를 store에 적어 둔다”이다.

기호 풀이 c_k = 원래 문맥 (대화 기록 · 문서 뭉치). $S(c; B_s)$ = 예산 B_s 로 그 문맥을 오픈라인 처리하는 절차이며 항상 인자를 괄호로 받는다. $\hat{c}$ = 그 산출물, 곧 **learned context**. $\text{wr}(E, \hat{c})$ = store에 쓰는 연산 — 덮어쓰기인가, 합집합인가, ADD/UPDATE/DELETE를 판단하는가가 E -경로 논문들을 가르는 축이다 (→ ch15).

한계 $\hat{c}$ 를 “요약”이라 부르지 않는다. 요약은 있는 것을 줄이는 일이지만 $S(\cdot; B_s)$ 는 없던 추론을 미리 수행한다. prefix cache와의 차이도 같은 자리에 있다 — cache는 같은 계산의 재사용이고 $\hat{c}$ 는 새 추론의 선행 수행이다. 이 한 줄이 이 책 전체에서 가장 자주 오해되는 지점이다.

표 0-2 — 쓰기 세 줄을 좌변으로 구분한다

식	좌변 (무엇이 바뀌는가)	시간척도	산출물의 물리적 형태	배포가 바뀌는가
(U-W)	W	토큰 · chunk t	fast weight 텐서	아니다
(U- Θ)	Θ	sleep 라운드 k	가중치 delta	그렇다
(U-E)	E	이벤트 · 세션 τ	텍스트 · 벡터 · 그 래프 레코드	아니다

자세한 것은 ch01 § 01.3.

0.5 비용은 항상 네 값 — $B_s, L_w, C_{\text{cap}}, \rho$

이 책은 어떤 절차든 아래 네 값으로 환산한다. 논문이 그 칸을 보고하지않았으면 “**논문에 없음**” 이라고 적고 추정치로 메우지 않는다.

표 0-3 — 비용 4 종과 그것이 대응하는 당신의 계기판

기호	무엇인가	단위	당신의 어휘	소유 장
B_s	질의 도착 전에 쓰는 계산 예산	FLOPs 또는 tokens	유티 시간 배치 job 의 청구서	ch01 § 01.5
L_w	wake 지연에 더 해지는 몫	ms, TTFT/ITL	SLA 항목	ch01 § 01.5
C_{cap}	남겨 두는 상태의 크기	bytes/세션또는 bytes/사용자	저장 sizing, checkpoint 크기	ch01 § 01.5
ρ	라운드를 반복할 때의 성능 손실	라운드당 손실	재실행의지속가능성	ch01 § 01.5

여기에 하나가 더 붙는다. B_t 는 **wake-time 계산 예산**, 즉 질의하나를 실제로 처리하는데 드는 추론 비용이다. sleep-time compute 의 목적을 한 줄로 쓰면 “ B_s 를 더 써서 B_t 를 줄이거나 정확도를올리는 것” 이다.

주의 L_w 는 정의의 시금석이다. 어떤 절차가 wake 지연을 늘린다면 그것은 sleep 이 아니라 그냥 wake-time 계산이다. **정의상 sleep 은 L_w 에 0 을 기여해야 한다.**

B_s 가 클수록 좋다는 주장은 그 자체로 성립하지 않는다. sleep 비용은 나중에 오는 질의들에게 나뉘어 실린다.

$$C_{\text{avg}} = C_{\text{wake}} + \frac{C_{\text{sleep}}}{N_q} \quad (\text{A})$$

직관 이 식이 말하는 것은 “질의 하나당 실제로 내는 값 = 그때그때 드는비용 + 미리 낸 비용을 나눈 몫”이다. 나누는 수 N_q 가 클수록 미리 낸값이 희석된다.

기호 풀이 N_q = 한 문맥을 몇 개의 질의가 공유하는가. 캐시의 reuse distance와 같은 자리의 값이다. N_q^* = 손익분기 - C_{avg} 가 sleep 없는 baseline과 같아지는 N_q 값이며, 이수를 넘겨야 sleep이 이긴다. κ = 생성 토큰을 입력 토큰의 몇 배로 셀 것인가. 비용 기호가 아니라 회계 가정이다.

비유 정기권이다. 한 번 크게 내고 탈 때마다 싸게 탄다. 몇 번 타야 본전인지가 N_q^* 다. **한계.** 정기권은 탄 횟수를 나중에 세면 되지만, sleep은 미리 결제해야 하고 그 문맥이 몇 번 재사용될지를 그 시점에 모른다. 게다가 본문의 판정에 따르면 실서빙에서 N_q 의 분포를 보고한 논문이 corpus에 없다 (→ ch14, ch25). 그래서 이 책의 모든 상각 논증은 조건부다.

시스템 모델링 관점 네 값은 서로 다른 계기판에 잡히고 하나의 수로 합쳐지지 않는다. B_s 는 야간 배치 클러스터의 GPU-시간으로, L_w 는 TTFT/ITL 대시보드로, C_{cap} 은 사용자 수를 곱한 스토리지 계획으로, ρ 는 “이 job을 매일 돌려도 되는가”라는 운영질문으로 나타난다. 그래서 “효율적”이라는 한 단어로 요약되는 주장은 이 책에서 미완성으로 취급된다. 특히 C_{cap} 은 corpus에서 가장 침묵당한 값으로, **상태 용량을 바이트로 보고 한 논문이 0편**이라는 것이 본문의 판정이다 (→ ch26).

시스템 모델링 관점 자릿수 감각 하나만 미리 갖고 가면 뒤 권이 쉬워진다. ch01의 예시 계산에 따르면, E-경로가 남기는 $\hat{c}$ 500 tokens는 세션당 약 **2 KB**다. 같은 자리에 Θ -경로가 rank 16 짜리 저랭크 delta를 남기면 $d = 4096 \cdot 32$ layer·2 projection 기준 사용자당 **16.8 MB**(bf16, 2 bytes/ele)다. 비는 약 **8,400 배**이고 사용자 100만 명이면 2 GB 대 16.8 TB다. 두 경로는 식 (A)로 보면 손익분기가 거의 같은데 (3.64 대 3.48) 저장 계획으로 보면 네 자릿수가 다르다 — 둘 다 이 책의 예시 계산이지 논문의 수치주장이 아니다. **이것이 비용을 하나가 아니라 4종으로 도입한 이유다.**

자세한 것은 ch01 §01.5, 그리고 통일 회계는 ch25, 용량 회계는 ch26.

0.6 첨자가 시간척도를 말한다

이 분야에서 가장 흔한 오독은 기호를 잘못 읽는 것이 아니라 **시간척도를 섞는** 것이다. 그래서 이 책은 첨자에 척도를 실어두었다.

표 0-4 — 다섯 개의 첨자

첨자	무엇을 세는가	얼마나 자주 도는가	소유 장
t	토큰 또는 chunk	추론 중 매 토큰	ch01 §01.3
s	SGD step	학습 job 안에서 배치 하나마다	ch02 §02.3

첨자	무엇을 세는가	얼마나 자주 도는가	소유 장
k	sleep 라운드	유희 시간마다 (하루 한 번, 세션 종료마다 등)	ch01 § 01.3
τ	세션	사용자가 돌아올 때	ch01 § 01.2
j	체크 · 세그먼트 인덱스	시간이 아니라 위치	ch17

비유 시계 다섯 개를 나란히 두고 각각 다른 속도로 돌린다고 보면 된다.
한계. 시계 바늘은 초 · 분 · 시가 고정배속으로 묶여 있지만 이 다섯은 그렇지 않다. 한 **sleep 라운드**가 몇토큰마다인지는 **논문이 아니라 시스템이 정한다**. 배속이 고정되어 있다고 가정하는 순간 비용 계산이 어긋난다.

k 는 **sleep 라운드 전용**이므로 **optimizer step** 에 쓰지 않는다. “step s 와 sleep 라운드 k 는 다르다” 를 구분하는 것 자체가 본문 Part I 의 교육 목표 중 하나다. 자세한 것은 ch02 § 02.3.

0.7 나머지 예약기호

본문에 나오는 나머지 글자는 이 표 하나로 끝난다. 막히면 여기로 돌아오면된다.

표 0-5 — 그 밖의 예약 기호

기호	무엇인가	주의할 점	소유 장
d	모델 폭		ch01
L	sequence 길이	층 수가 아니다	ch01
L_{layer}	층 수	L 과 반드시 구분	ch01
n	store 에 저장된 항목 수	체크 인덱스로 쓰지 않는다 (그것은 j)	ch10
C	chunk 크기	용량은 언제나 C_{cap}	ch01
r	LoRA rank	보상은 R 로 쓴다	ch03 § 03.3
$\mathcal{S}$	LoRA 를 적용하는 행렬 집합 (예: $\{W_q, W_v\}$)	칼리그래픽 $\mathcal{S}$ 전용	ch03 § 03.4
R	강화학습의 보상		ch05 § 05.1
Π	지속학습 성능 행렬. Π_{ij} = 과제 j 까지 배운 뒤 과제 i 의 성능	ACC·BWT·FWT 가 전부 여기서 나온다	ch07
ω	replay 혼합비 (한 배치에서 과거 데이터의 비율)		ch07
z	표본 · 후보	s 가 step 으로 예약되어 있어서	ch06 § 06.4
$\mathcal{M}(\cdot; W)$	memory 읽기 함수		ch01

주의 바이트를 말할 때는 언제나 dtype 을 병기한다 (“bf16 기준 2 bytes/elem”). 이 규칙 하나가 C_{cap} 논의의절반을 지탱한다 — 같은 파라미터 수가 int8(1 byte/elem) 과 fp32(4 bytes/elem) 사이에서 네 배 벌어지므로, dtype 을 밝히지 않은 바이트 수는 비교에 쓸 수 없다.

자세한 것은 본문 용어집 § 91.4.

0.8 절대 헛갈리면 안 되는 세쌍

같은 글자가 두 뜻으로 보이는 자리는 셋뿐이고, 셋 다 구분 규칙이기 때문이다.

표 0-6 — 세 쌍과 구분 규칙

충돌	구분 규칙
C 대 C_{cap}	맨 C 는 chunk 크기 . 상태용량은 언제나 아래첨자를 단 C_{cap}
S_t 대 $S(c; B_s)$	S_t 는 (U-W) 의 momentum 이며 항상시간 첨자를 단다. $S(c; B_s)$ 는 (U-E) 의 오프라인 처리 절차 이며 항상 인자를 괄호로 받는다. 첨자도 괄호도 없는 맨 S 는 쓰지 않는다
η_t 대 η_Θ	η_t 는 토큰 척도의 inner learning rate, η_Θ 는 sleep 라운드 척도의 learning rate

이 셋을 섞어 쓰는 것은 본문에서 결함으로 처리된다. 독자 입장에서도 마찬가지다 — 세쌍만 지키면 본문의 어떤 식도 오독되지 않는다. 자세한 것은 ch01 § 01.3·§ 01.5.

0.9 세 경로와 판별식

같은 이름 아래 들어온 절차를 층으로 갈라 부르는 이름이 **경로**다.

- **E-경로** — (U-E) 만 실행한다. Θ 와 W 는 그대로다.
- **W-경로** — (U-W) 를 wake 가 아니라 **오프라인에 추가로** 실행한다.
- **Θ -경로** — (U- Θ) 를 실행한다. **배포 아티팩트가 바뀐다**. 이것이 나머지 둘과의 결정적 차이다.

이 셋을 parametric/non-parametric 의 이분법으로 부르지 않는다. 그 이분법은 W-경로를 어느 쪽에도 놓지 못한다.

어떤 절차가 애초에 이 범주에 속하는지는 네 조건으로 판정한다.

핵심 판별식. (1) **시계**: 상태를 바꾸는 연산이 질의 도착전에 실행되는가. (2) **예산**: 그 연산이 $B_s > 0$ 을 소비하는가. (3) **대상**: $\Theta \cdot W \cdot E$ 중 어느 한 층의 상태를 실제로 바꾸는가. (4) **소비자**: 바뀐 상태가 그것을 유발한 질의가 아니라 이후 질의에 쓰이는가. 넷을 모두 만족해야 한다.

비유 네 관문짜리 세관 검사다. 하나라도 통과하지 못하면 인접 문헌이지 이범주가 아니다.

한계. 세관은 신고서와 물건을 함께 보지만 이 판별식은 **논문의 자기 설명을 전혀 입력으로 받지 않는다**. 보는 것은 갱신식과 시계뿐이고, 성능도 자기 규정도 묻지 않는다. 그래서 스스로를 sleep 이라 부르는 논문이 탈락하고 그렇게 부르지 않는 논문이 통과하는 일이 실제로 일어난다.

본문이 corpus 전체에 이 넷을 적용한 결과는 **통과 9편 + 경계 1편 + 불충족 19편**이다. 이 책의 척추 논지 — **이름이 기계보다 넓다** — 가여기서 나온다. 자세한 것은 ch01 § 01.4(도구), ch11 § 11.2 – § 11.3(적용과 전수 결과).

0.10 본문에 붙어 있는 표시들

본문은 논문의 주장과 자기 해설과 자기 판단을 섞지 않는다. 표시를 알아두면 어느 문장이 누구의 것인지 즉시 읽힌다.

표 0-7 — 본문의 귀속 표시

표시	누구의 말인가
위치 병기 ([SEAL §4.2] 등)	논문의 주장 · 보고. 원 논문 절 번호는 언제나 논문 이름과 함께 쓴다
[해설] 블록	논문 내용을 독자의 어휘로 옮긴 이 책의 재서술
[평가] 블록	논문과 다른 이 책의 판단
[본서 관찰] · [본서 산술] · [본서 추론] · [본서 판단]	문장 하나단위의 귀속. 논문 수치에서 이 책이 계산해 낸 값에는 [본서 산술] 이 의무다
“ 논문에 없음 ”	비용 4 종 표의 빈 칸. 추정치로 메우지 않는다

맨 §3.1 처럼 논문 이름 없이 쓰인 절 번호는 언제나 **이 책의 절**을 가리킨다. 자세한 것은 본문 용어집 §91.9.

요약 - 이 책의 한 문장: **질의가 오기 전에 계산을 미리 해서 상태를 바꿔둔다**. 문제는 “상태”가 셋이라는 것이다. - 상태 세 층은 Θ (배포가중치) · W (요청에 딸린 고정 크기 상태) · E (모델 밖 저장소) 이고, 이들을 가르는 기준은 크기가 아니라 누가 소유하고 언제 사라지는가다. - 읽기는 식 (R) 하나, 쓰기는 $(U-W) \cdot (U-\Theta) \cdot (U-E)$ 셋이다. **세식의 차이는 전부 좌변에 있다** — 어떤 절차를 이해했다는 것은 그 좌변을 적을 수 있다는 뜻이다. - 비용은 언제나 네 값이다: B_s (미리 쓰는 계산) · L_w (wake 지연) · C_{cap} (남기는 바이트) · ρ (반복했을 때의 손실). 식 (A)의 손익분기 N_q^* 는 상각의 도구이고, 네 값은 하나로 합쳐지지 않는다. - 침자가 시간척도를 말한다: t (토큰) · s (SGD step) · k (sleep 라운드) · τ (세션). 섞으면 비용 계산이 어긋난다. - 헛갈릴 자리는 셋뿐이다: C 대 C_{cap} , S_t 대 $S(c; B_s)$, η_t 대 η_Θ . 구분 규칙은 전부 침자와 괄호의 유무다.

다음 권 예고. 이 권은 글자만 쥐여 주었고 그 글자들이 가리키는 절차 — $\text{gen}(\mathcal{H}_k; B_s)$ 가 실제로 무슨 job 이고, 한 라운드를 “돈다”는 것이 실무에서 무엇인지 — 는 열지 않았다.

다음 권이그 블랙박스를 연다. 그 전에 본문을 펼친다면 ch01 부터 읽으면 되고, 기호가 막히면 언제든 이 권의 § 0.5 와 § 0.7 로 돌아오면 된다.